# 📘 MODUL LENGKAP: MASTERY BACKEND LARAVEL API

**Panduan** Belajar Pemrograman Web (Backend) untuk SMK RPL **Durasi:** 23 Pertemuan (1 Semester) | **Fase:** F

## 🗺️ PETA PERJALANAN BELAJAR (ROADMAP 23 PERTEMUAN)

Agar kamu tidak tersesat, ini adalah rute perjalanan kita selama 1 semester:

**FASE 1: FONDASI & DASAR (Pertemuan 1-7)**

- **Minggu 1:** Konsep API, REST, & Instalasi Tools.
- **Minggu 2:** Bedah Struktur Laravel & Konfigurasi Environment.
- **Minggu 3:** Database Migration (Single Table).
- **Minggu 4:** Eloquent Model & Seeding Data Dummy.
- **Minggu 5:** Routing & Controller Dasar.
- **Minggu 6:** Logika CRUD Bagian 1 (Read & Create).
- **Minggu 7:** Logika CRUD Bagian 2 (Update, Delete, Validasi).

**FASE 2: RELASI DATABASE & FITUR LANJUTAN (Pertemuan 8-15)**

- **Minggu 8:** Konsep Relasi (Foreign Key) & Migration Tabel Berelasi.
- **Minggu 9:** Eloquent Relationship (`BelongsTo`, `HasMany`).
- **Minggu 10:** CRUD Data Berelasi (Simpan Produk dengan Kategori).
- **Minggu 11:** Autentikasi API (Register & Login) dengan Laravel Sanctum.
- **Minggu 12:** Middleware & Proteksi Route (Hanya User Login).
- **Minggu 13:** API Resources (Standarisasi Format JSON).
- **Minggu 14:** Upload File (Gambar Produk).
- **Minggu 15:** Evaluasi Tengah Semester (Mini Project).

**FASE 3: PROFESIONAL & FINAL PROJECT (Pertemuan 16-23)**

- **Minggu 16:** Fitur Pencarian (Search) & Pagination.
- **Minggu 17:** Error Handling (Menangani Pesan Error Cantik).
- **Minggu 18:** Perencanaan Final Project (Desain DB & Alur).
- **Minggu 19-21:** SPRINT CODING (Pengerjaan Proyek Mandiri).
- **Minggu 22:** Deployment (Upload ke Hosting).
- **Minggu 23:** Presentasi Final & Review.

# BAGIAN 1: TEORI DASAR (Minggu 1)

## 1.1 Apa itu API?

Bayangkan kamu di restoran:

1. **Kamu (Frontend):** Melihat menu di meja.
2. **Dapur (Backend):** Tempat masak bahan makanan (Database).
3. **Pelayan** (API): Kamu pesan ke pelayan, pelayan ke dapur, lalu pelayan bawa makanan ke kamu.

**API** (Application Programming **Interface)** adalah jembatan penghubung antara aplikasi klien (HP/Web) dengan server.

## 1.2 Aturan Main (HTTP Verbs)

Saat kita menyuruh API, kita pakai kode kata kerja:

- **GET:** "Ambilkan data" (Contoh: Lihat daftar barang).
- **POST:** "Simpan data baru" (Contoh: Tambah barang).
- **PUT/PATCH:** "Ubah data" (Contoh: Ganti harga barang).
- **DELETE:** "Hapus data".

## 1.3 Tools Wajib

1. **XAMPP:** Server lokal (PHP & MySQL).
2. **Composer:** "App Store"-nya PHP untuk download library.
3. **VS Code:** Tempat ngoding.
4. **Postman:** Aplikasi untuk ngetes API (karena kita belum punya tampilan web).

# BAGIAN 2: PERSIAPAN DAPUR (Minggu 2)

## 2.1 Instalasi Laravel

Buka terminal (CMD/Git Bash), masuk ke folder `htdocs / www`, ketik:

*composer create-project laravel/laravel my-api*

Masuk ke folder: `cd my-api`

## 2.2 Koneksi Database (.env)

1. Nyalakan Apache & MySQL di XAMPP.

2. Buka `localhost/phpmyadmin`, buat database baru: `db_toko_online`.
3. Di VS Code, buka file `.env`, ubah bagian ini:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_toko_online  <-- Ganti ini
DB_USERNAME=root
DB_PASSWORD=
```

# BAGIAN 3: DATABASE SINGLE TABLE (Minggu 3-4)

Kita mulai dengan tabel sederhana: **Categories** (Kategori Produk).

## 3.1 Membuat Migration & Model

Di terminal VS Code:

```
php artisan make:model Category -m
```

*(Huruf* -m artinya *sekalian buat file migration)*

## 3.2 Desain Tabel Kategori

Buka file di `database/migrations/xxxx_xx_xx_create_categories_table.php`.

```
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name'); // Nama Kategori (Elektronik, Pakaian)
        $table->timestamps();
    });
}
```

Llalu jalankan: `php artisan migrate`

## 3.3 Setup Model (Agar Aman)

Buka `app/Models/Category.php`. Tambahkan `$fillable`.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    use HasFactory;

    // Kolom yang boleh diisi user
    protected $fillable = ['name'];
}
```

# BAGIAN 4: BASIC CRUD (Minggu 5-7)

Kita buat fitur Create, Read, Update, Delete untuk Kategori.

## 4.1 Buat Controller

php artisan make:controller Api/CategoryController

## 4.2 Kode Lengkap Controller

Buka `app/Http/Controllers/Api/CategoryController.php`.

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class CategoryController extends Controller
{
    // GET: Ambil Semua Kategori
    public function index()
    {
```

```
    $categories = Category::all();
    return response()->json([
        'status' => true,
        'message' => 'List Kategori',
        'data' => $categories
    ], 200);
}

// POST: Tambah Kategori
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'name' => 'required|unique:categories,name'
    ]);

    if ($validator->fails()) {
        return response()->json(['errors' => $validator->errors()], 422);
    }

    $category = Category::create([
        'name' => $request->name
    ]);

    return response()->json([
        'status' => true,
        'message' => 'Kategori dibuat',
        'data' => $category
    ], 201);
}

// GET: Detail 1 Kategori
public function show($id)
{
    $category = Category::find($id);
    if (!$category) return response()->json(['message' => 'Tidak ditemukan'], 404);

    return response()->json(['data' => $category], 200);
}

// PUT: Update Kategori
public function update(Request $request, $id)
{
    $category = Category::find($id);
    if (!$category) return response()->json(['message' => 'Tidak ditemukan'], 404);

    $category->update(['name' => $request->name]);

    return response()->json(['message' => 'Berhasil update', 'data' => $category], 200);
```

```
    }

    // DELETE: Hapus Kategori
    public function destroy($id)
    {
        $category = Category::find($id);
        if (!$category) return response()->json(['message' => 'Tidak ditemukan'], 404);

        $category->delete();
        return response()->json(['message' => 'Berhasil dihapus'], 200);
    }
}
```

## 4.3 Daftarkan Route

Buka `routes/api.php`:

```
use App\Http\Controllers\Api\CategoryController;

Route::apiResource('/categories', CategoryController::class);
```

*Sekarang, silakan tes di Postman (GET, POST, PUT, DELETE) ke [http://127.0.0.1:8000/api/categories](http://127.0.0.1:8000/api/categories).*

*****CATATAN******

Jika tidak ditemukan file routes/api.php (terutama untuk laravel versi terbaru), jalankan perintah:
*php artisan install:api*

# BAGIAN 5: RELASI DATABASE (Minggu 8-10)

Sekarang kita masuk level menengah. Produk pasti punya Kategori. **Relasi:** Satu Kategori punya banyak Produk (*One to Many*).

## 5.1 Buat Migration Produk dengan Foreign Key

php artisan make:model Product -m

Buka file migration products. Perhatikan cara kita menyambungkan ke tabel categories.

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        // Foreign Key ke tabel categories
        $table->foreignId('category_id')->constrained('categories')->onDelete('cascade');

        $table->string('name');
        $table->integer('price');
        $table->integer('stock');
        $table->text('description')->nullable();
        $table->timestamps();
    });
}
```

Jalankan: `php artisan migrate`

## 5.2 Setup Relasi di Model (Eloquent)

Agar Laravel tahu hubungannya, kita edit kedua model.

**File:** `app/Models/Category.php` (Tambahkan ini)

```
public function products()
{
    // Satu kategori punya banyak produk
    return $this->hasMany(Product::class);
}
```

**File:** `app/Models/Product.php` (Kode Lengkap)

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;

    protected $fillable = [
        'category_id', 'name', 'price', 'stock', 'description', 'image'
```

```
    ];

    public function category()
    {
        // Produk ini milik satu kategori
        return $this->belongsTo(Category::class);
    }
}
```

## 5.3 Controller Produk dengan Relasi

Kita buat `ProductController` yang bisa menyimpan produk sekaligus ID kategorinya.

```
php artisan make:controller Api/ProductController
```

Isi `app/Http/Controllers/Api/ProductController.php`:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class ProductController extends Controller
{
    public function index()
    {
        // 'with' digunakan untuk memuat data kategori sekaligus (Eager Loading)
        $products = Product::with('category')->latest()->get();

        return response()->json(['data' => $products], 200);
    }

    public function store(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'category_id' => 'required|exists:categories,id', // Pastikan ID kategori ada
            'name' => 'required',
            'price' => 'required|numeric',
            'stock' => 'required|numeric',
        ]);
```

```
        if ($validator->fails()) return response()->json(['errors' => $validator->errors()], 422);

        $product = Product::create($request->all());

        return response()->json(['message' => 'Produk tersimpan', 'data' => $product], 201);
    }

    // ... (Implementasikan Show, Update, Destroy mirip dengan CategoryController)
}
```

Daftarkan di `routes/api.php`:

```
use App\Http\Controllers\Api\ProductController;
Route::apiResource('/products', ProductController::class);
```

# BAGIAN 6: AUTENTIKASI (Minggu 11-12)

Kita akan membuat fitur Login & Register menggunakan **Laravel Sanctum**.

## 6.1 Instalasi Sanctum

1. `php artisan install:api` (Untuk Laravel 11, pilih Yes jika ditanya migration).
2. `php artisan migrate` (Untuk membuat tabel `users` dan `personal_access_tokens`).

## 6.2 Auth Controller

php artisan make:controller Api/AuthController

Buka `app/Http/Controllers/Api/AuthController.php`. Kode Lengkap:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Validator;
```

```php
class AuthController extends Controller
{
    // REGISTER
    public function register(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'name' => 'required',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8'
        ]);

        if ($validator->fails()) return response()->json($validator->errors(), 422);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password)
        ]);

        return response()->json(['message' => 'Register Berhasil', 'data' => $user], 201);
    }

    // LOGIN
    public function login(Request $request)
    {
        if (!Auth::attempt($request->only('email', 'password'))) {
            return response()->json(['message' => 'Email atau Password salah'], 401);
        }

        $user = User::where('email', $request->email)->firstOrFail();

        // Membuat Token (Tiket Masuk)
        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'message' => 'Login Berhasil',
            'access_token' => $token,
            'token_type' => 'Bearer'
        ], 200);
    }

    // LOGOUT
    public function logout(Request $request)
    {
        // Hapus token yang sedang dipakai
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Logout Berhasil'], 200);
    }
```

}

## 6.3 Proteksi Route (Middleware)

Buka `routes/api.php`. Kita atur agar **Kategori** dan **Produk** hanya bisa dimanipulasi oleh user yang login.

```
use App\Http\Controllers\Api\AuthController;

// Route Public (Bisa diakses siapa saja)
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Route Private (Harus bawa Token)
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);

    // CRUD Produk & Kategori kita masukkan sini
    Route::apiResource('/categories', CategoryController::class);
    Route::apiResource('/products', ProductController::class);
});
```

*Cara* Tes di Postman: Login dulu -> Copy Token -> Ke Tab Authorization -> Pilih Bearer Token -> Paste Token -> *Coba akses Produk.*

# BAGIAN 7: UPLOAD GAMBAR (Minggu 14)

Kita update `ProductController` agar bisa upload gambar.

## 7.1 Persiapan Storage

Ketik di terminal:

```
php artisan storage:link
```

## 7.2 Update Controller (Method Store)

Edit `ProductController.php` pada method `store`.

```
public function store(Request $request)
{
    // ... Validasi sebelumnya ...
    $validator = Validator::make($request->all(), [
        'image' => 'required|image|mimes:jpeg,png,jpg|max:2048', // Tambahan validasi gambar
        // ... validasi lain ...
    ]);

    // Proses Upload
    $image = $request->file('image');
    $image->storeAs('public/products', $image->hashName());

    // Simpan ke DB
    $product = Product::create([
        'image' => $image->hashName(), // Simpan nama filenya saja
        'category_id' => $request->category_id,
        'name' => $request->name,
        'price' => $request->price,
        'stock' => $request->stock,
        'description' => $request->description
    ]);

    return response()->json(['data' => $product], 201);
}
```

# BAGIAN 8: FITUR ADVANCED (Minggu 16)

## 8.1 Search & Pagination

Daripada menampilkan 1000 produk sekaligus, kita bagi per halaman. Edit `index` di `ProductController.php`.

```
public function index(Request $request)
{
    // Mulai query
    $query = Product::with('category');

    // Jika ada pencarian nama (?name=sepatu)
    if ($request->has('name')) {
        $query->where('name', 'like', '%' . $request->name . '%');
    }
```

```
    // Pagination (5 data per halaman)
    $products = $query->latest()->paginate(5);

    return response()->json($products, 200);
}
```

# BAGIAN 9: FINAL PROJECT BRIEF (Minggu 18-23)

## Studi Kasus: "Sistem Inventory Sekolah"

**Tugas:** Buatlah API lengkap dengan spesifikasi berikut:

1. **Auth:** Login untuk Admin dan Petugas.
2. **Tabel:** * `Users` (Admin/Petugas)
    - `Items` (Barang)
    - `Categories` (Kategori Barang)
    - `Transactions` (Peminjaman/Pengembalian Barang).
3. **Relasi:** * Satu Kategori -> Banyak Barang.
    - Satu Barang -> Banyak Transaksi.
    - Satu User -> Banyak Transaksi.
4. **Fitur:** Upload Foto Barang, Search Barang, Riwayat Peminjaman.

**Kriteria Penilaian:**

- Kode berjalan tanpa error (40%).
- Implementasi Relasi Database benar (20%).
- Fitur Autentikasi berjalan (20%).
- Struktur kode rapi & ada komentar (10%).
- Dokumentasi Postman (10%).

*Selamat Belajar! Jangan ragu untuk mencoba